

Tarea 4

Índice

1. Introducción	1
2. Implementaciones	1
2.1. Secuencial	1
2.1.1. Compilación del programa	2
2.1.2. Ejemplos	2
2.2. OpenMPI	5
2.2.1. Compilación del programa	6
2.2.2. Ejemplos	6
3. Análisis de resultados	8
4. Conclusiones	9

1. Introducción

La difusión es un proceso natural mediante el cual una magnitud física, como el calor o la concentración de una sustancia, se propaga en el espacio. Este fenómeno ocurre en diversos contextos: transferencia térmica en materiales, dispersión de contaminantes en fluidos, y transporte de nutrientes en sistemas biológicos. Su modelado es crucial para comprender y controlar estos sistemas en disciplinas como la ingeniería, física, química y ciencias ambientales.

Supongamos que $\{u_{i,j}\}$ representa la temperatura de una malla bidimensional, cuyo valor inicial es cero en todos los puntos, excepto el nodo central, donde $u_{\frac{N}{2}, \frac{N}{2}} = 100$ en todo momento. El esquema iterativo que modela la propagación de calor está dado por

$$u_{i,j}^{\text{new}} = u_{i,j} + \alpha(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j})$$

donde $\alpha = 0.1$. Este esquema indica que el valor en cada punto (sin contar las fronteras), en la nueva iteración, se actualiza como una combinación ponderada de sus vecinos más cercanos.

2. Implementaciones

2.1. Secuencial

Para implementar el esquema de manera secuencial, realizamos el siguiente bucle:

```
for (int j = 1; j < N - 1; ++j)
{
    for (int i = 1; i < N - 1; ++i)
    {
        int idx = i + j * N;

        u_next[idx] = (u[idx] * self_cooling) +
                      alpha * (u[idx + 1] + u[idx - 1] + u[idx + N] + u[idx - N]);
    }
}

// constant center heat
u_next[center_idx] = u0;

std::swap(u, u_next);
```

Listado 1: Snippet de actualizaciones realizadas para cada pixel.

La constante `self_cooling = 1 - 4*alpha` representa la cantidad de calor que pierde la partícula cada iteración, que es propagada a las partículas adyacentes. Dado el enunciado, ignoramos los bordes, por lo que nuestro bucle ignora el primer y último valor.

2.1.1. Compilación del programa

Usamos

```
g++ diffusion_seq.cpp -O3 -o seq
```

para compilar. Notemos que usamos la bandera de optimización `-O3`. Esto se debe a que como imprimimos los resultados de cada 10 iteraciones, el tiempo total de ejecución es demasiado alto. Entonces, decidimos utilizar las optimizaciones del compilador para reducir el tiempo de ejecución aproximadamente 10x. Para ejecutarlo, usamos

```
./seq [u0] [alpha] [N] [MAX_ITER] > file.csv
```

Por defecto, los valores son $u_0 = 100$, $\alpha = 0.1$, $N = 50$, $\text{MAX_ITER} = 10000$.

2.1.2. Ejemplos

El siguiente ejemplo muestra el problema particular.

```
./seq 100 0.1 20000
```

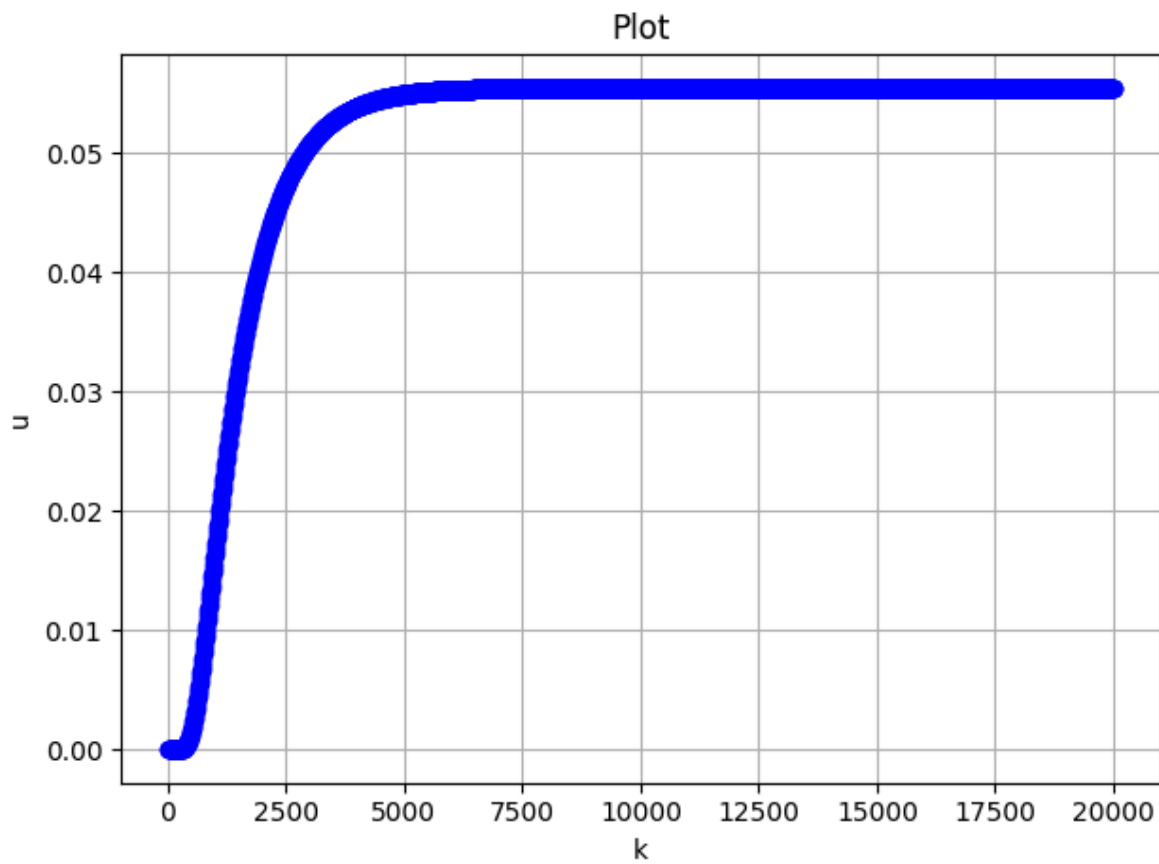


Figura 1: Valores de $u[2][2]$ para $N = 50$ en cada iteración. Resultados de programa secuencial.

```
./seq 100 0.1 500 200000
```

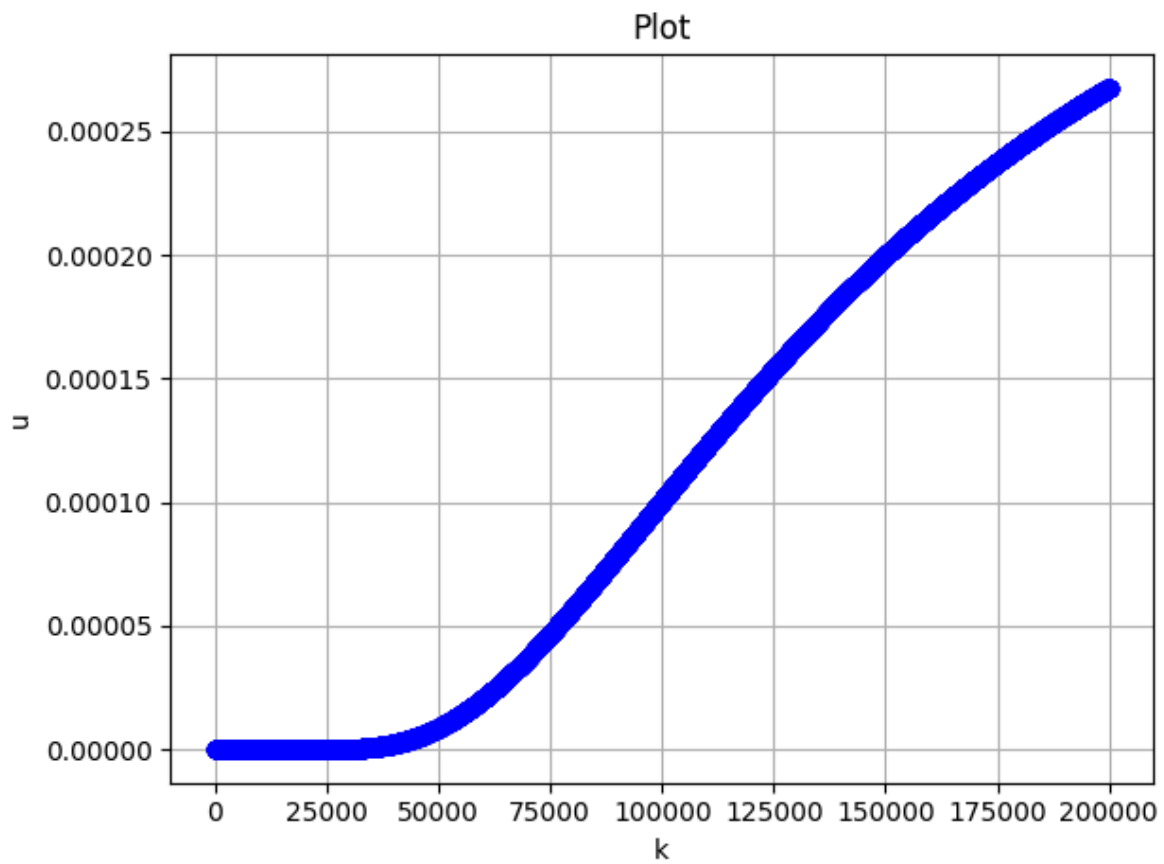


Figura 2: Valores de $u[2][2]$ para $N = 500$ en cada iteración. Resultados de programa secuencial.

```
./seq 100 0.1 1000 200000
```

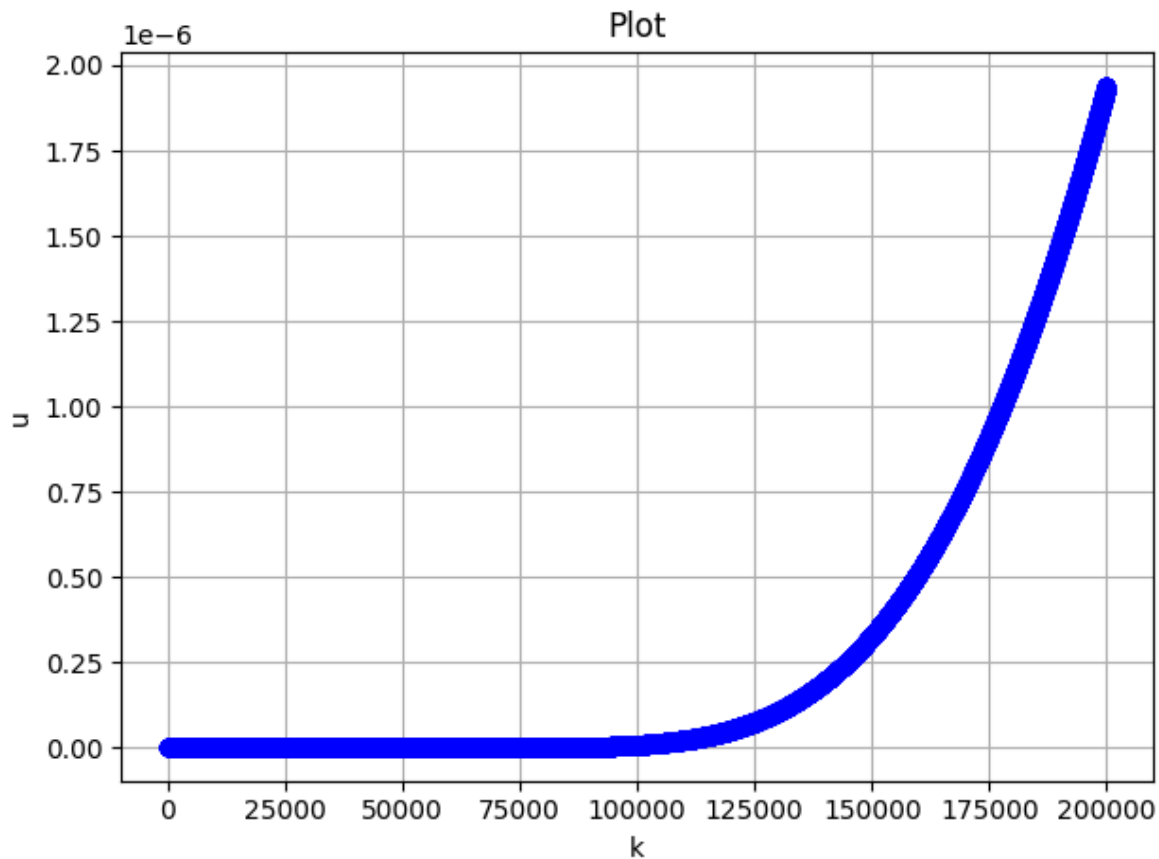


Figura 3: Valores de $u[2][2]$ para $N = 1000$ en cada iteración. Resultados de programa secuencial.

Esta última no parece tener el mismo comportamiento que las previas. Sin embargo, esto se debe a la cantidad de iteraciones que le permitimos. Es de esperar que en algún punto se vuelva una temperatura estable.

2.2. OpenMPI

Para la implementación en MPI, usamos `MPI_Type_vector`:

```
MPI_Type_vector(N, 1, local_width, MPI_DOUBLE, &columntype);
```

Esto nos permite esto nos genera las franjas (con las celdas ghost) que manejará cada procesador. Luego, los cálculos son idénticos al programa secuencial. Lo único que cambia es el hecho de que debemos comunicarnos con los procesos adyacentes. Esto lo logramos mediante

```
MPI_Sendrecv(&u[1], 1, columntype, left_neighbor, 0, &u[col_size + 1], 1,
columntype, right_neighbor, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Sendrecv(&u[col_size], 1, columntype, right_neighbor, 1, &u[0], 1, columntype,
left_neighbor, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Listado 8: La primera línea envía la info `&u[1]` al vecino izquierdo, y recibe `&u[col_size+1]` del mismo vecino. La segunda hace algo similar con los nodos de la parte derecha.

Esta sincronización permite que las franjas tengan la información correcta de los nodos fantasma.

2.2.1. Compilación del programa

Para compilar la versión OpenMPI, usamos

```
mpicxx diffusion_mpi.cpp -O3 -o mpi
```

Notemos nuevamente que usamos las optimizaciones del compilador. Esto es para no darle ventaja a la versión secuencial. Ambos imprimen los resultados, por lo que ralentiza el tiempo total.

2.2.2. Ejemplos

El siguiente ejemplo muestra el problema particular, usando 2 procesos

```
mpirun -n 2 ./mpi 100 0.1 50 20000
```

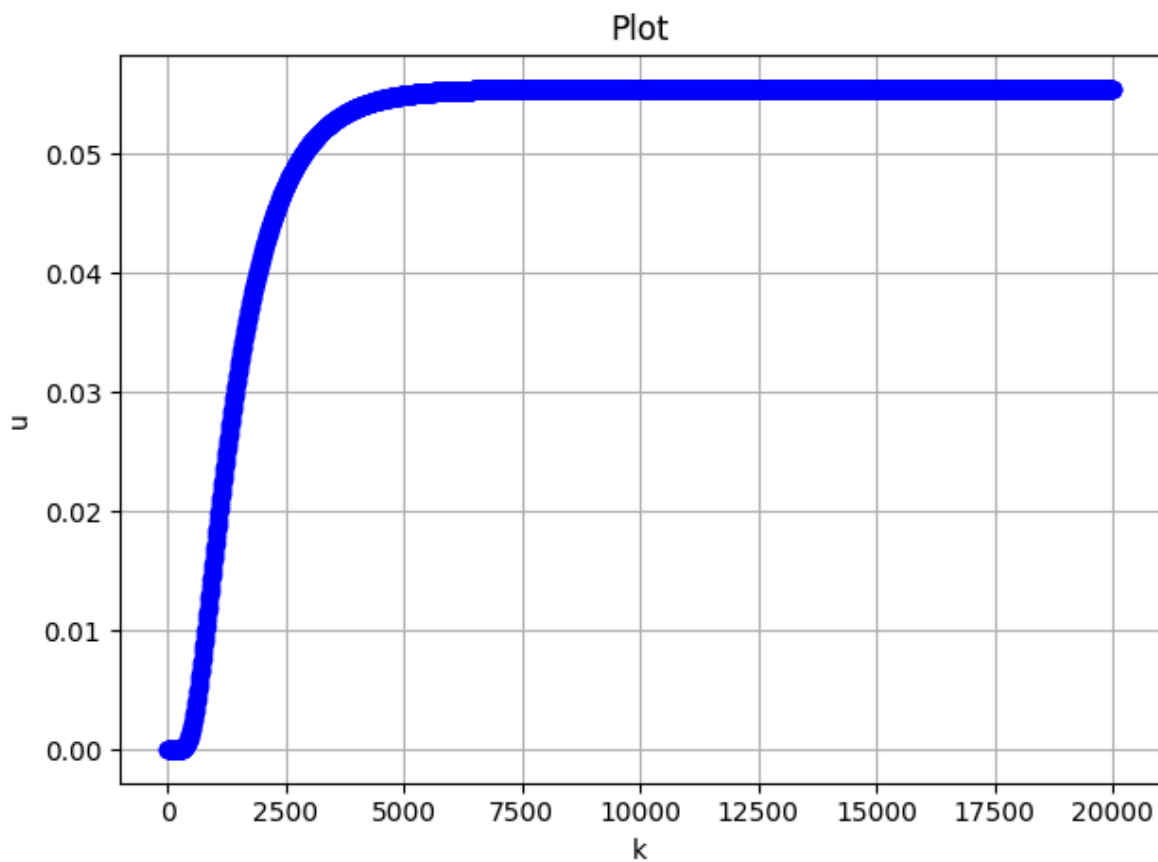


Figura 4: Valores de $u[2][2]$ para $N = 50$ en cada iteración. Resultados usando MPI.

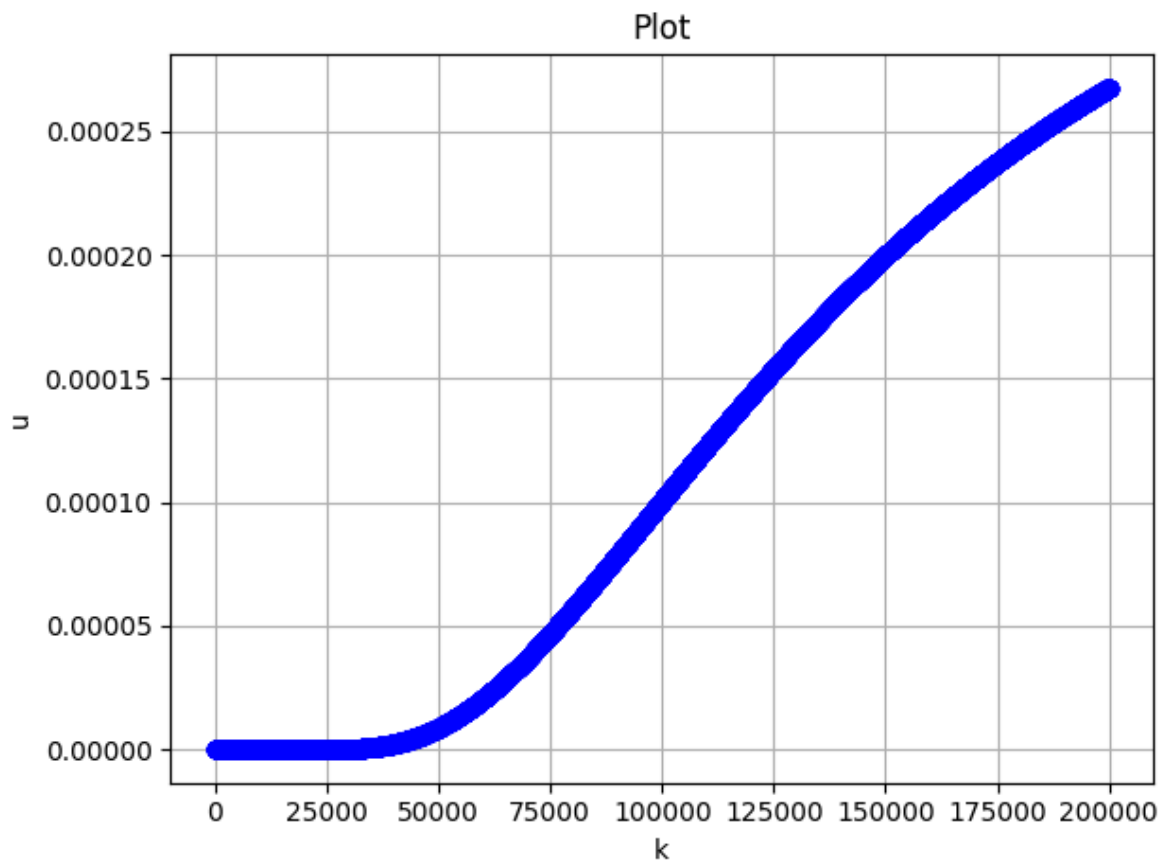


Figura 5: Valores de $u[2][2]$ para $N = 500$ en cada iteración. Resultados usando MPI.

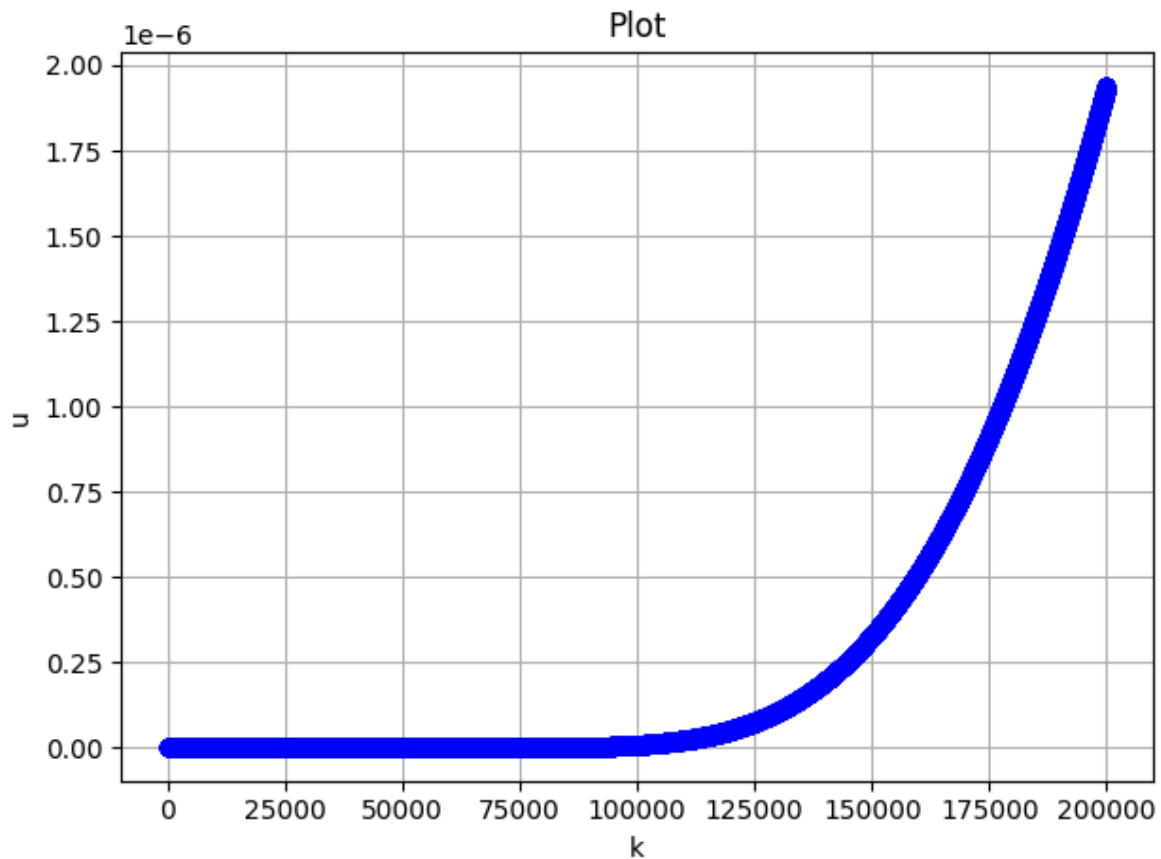


Figura 6: Valores de $u[2][2]$ para $N = 1000$ en cada iteración. Resultados usando MPI.

3. Análisis de resultados

Para la comparación de resultados, usamos el equipo de supercómputo El Insurgente. El slurm utilizado fue

```
#!/bin/bash

#SBATCH --partition=C1Mitad1
#SBATCH --job-name=pedro_llerenas
#SBATCH --ntasks-per-node=12
#SBATCH --ntasks=4
#SBATCH --output=result.log
#SBATCH --mem=0
#SBATCH --time=0

mpirun.openmpi -np ${SLURM_NTASKS} mpi 100 0.1 50 200000
```

Donde se modificaron los valores de `ntasks` y el número de divisiones del cuadro. La siguiente tabla resume los resultados obtenidos para cada ejecución.

N	Versión	Tiempo (ms)	Speedup	Eficiencia
50	Secuencial	449.934	1	1
	MPI (-n 2)	1387.83	0.324	0.162
	MPI (-n 4)	1432.16	0.314	0.079
	MPI (-n 8)	1275.8	0.353	0.044
	MPI (-n 16)	15606.6	0.029	0.002
500	Secuencial	42320.2	1	1
	MPI (-n 2)	90902.7	0.466	0.233
	MPI (-n 4)	48023	0.881	0.220
	MPI (-n 8)	29586.2	1.430	0.179
	MPI (-n 16)	33428.8	1.266	0.079
1000	Secuencial	241849	1	1
	MPI (-n 2)	361053	0.670	0.335
	MPI (-n 4)	185440	1.304	0.326
	MPI (-n 8)	101597	2.380	0.298
	MPI (-n 16)	82460.4	2.933	0.183

Tabla 1: Rendimiento comparativo entre ejecución secuencial y distinto número de procesadores.

4. Conclusiones

Los tiempos obtenidos sugieren que hay una relación entre N y el número de procesos utilizados. Utilizar demasiados procesos para un N pequeño resulta contraproducente, ya que la cantidad de comunicación que se debe hacer sobrepasa el tiempo que toma realizar los cálculos. Entonces, se debe experimentar con el número de procesadores para ver dónde empeora la eficiencia.

Otra posible mejora es realizar las particiones por proceso de manera horizontal. Esto permite que la mayor parte de las operaciones sean realizadas en un bloque contiguo de memoria, evitando problemas de cache.